UNIQ ID: E0124092

```
!pip install gymnasium[toy_text,classic_control] -q


[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

1. Print the total number of environments

```
import gymnasium as gym

# Retrieve all registered environments
all_envs = gym.envs.registry.keys()

# Count the total number of environments
total_envs = len(all_envs)
print(f"Total number of environments: {total_envs}")

Total number of environments: 63
```

2. Print all environment names

```
import gymnasium as gym

# Retrieve and sort all registered environment names
env_names = sorted(gym.envs.registry.keys())

for name in env_names:
    print(name)

Acrobot-v1
Ant-v2
Ant-v3
Ant-v4
Ant-v5
BipedalWalker-v3
BipedalWalkerHardcore-v3
Blackjack-v1
CarRacing-v3
CartPole-v0
CartPole-v1
CliffWalking-v1
CliffWalkingSlippery-v1
FrozenLake-v1
FrozenLake8x8-v1
GymV21Environment-v0
GymV26Environment-v0
HalfCheetah-v2
HalfCheetah-v3
HalfCheetah-v4
HalfCheetah-v5
```

```
Hopper-v2
Hopper-v3
Hopper-v4
Hopper-v5
Humanoid-v2
Humanoid-v3
Humanoid-v4
Humanoid-v5
HumanoidStandup-v2
HumanoidStandup-v4
HumanoidStandup-v5
InvertedDoublePendulum-v2
InvertedDoublePendulum-v4
InvertedDoublePendulum-v5
InvertedPendulum-v2
InvertedPendulum-v4
InvertedPendulum-v5
LunarLander-v3
LunarLanderContinuous-v3
MountainCar-v0
MountainCarContinuous-v0
Pendulum-v1
Pusher-v2
Pusher-v4
Pusher-v5
Reacher-v2
Reacher-v4
Reacher-v5
Swimmer-v2
Swimmer-v3
Swimmer-v4
Swimmer-v5
Taxi-v4
Walker2d-v2
Walker2d-v3
Walker2d-v4
Walker2d-v5
phys2d/CartPole-v0
phys2d/CartPole-v1
phys2d/Pendulum-v0
tabular/Blackjack-v0
tabular/CliffWalking-v0
```

3. Explore different environments and define their MDP

For each environment we will:

- Load the environment
- Display the action space and observation space
- Explore MDP components (State, Action, Transition, Reward)

```python
def explore_env(env_name, seed=42):
    print(f"\n{'='*60}")
    print(f"Environment: {env_name}")
```

```
    print(f"{'='*60}")

    env = gym.make(env_name)

    # Action and observation space
    print(f"Action Space: {env.action_space}")
    print(f"Observation Space: {env.observation_space}")

    # Reset to get the initial state
    state, info = env.reset(seed=seed)
    print(f"Initial State: {state}")

    # Take a sample action to explore transition & reward
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)

    print(f"Sample Action Taken: {action}")
    print(f"Next State (Transition): {next_state}")
    print(f"Reward: {reward}")
    print(f"Terminated: {terminated}, Truncated: {truncated}")
    print(f"Info: {info}")

    env.close()
    return env
```

3.1 CartPole

```
explore_env("CartPole-v1")
```

```
============================================================
Environment: CartPole-v1
============================================================
Action Space: Discrete(2)
Observation Space: Box([-4.8               -inf -0.41887903        -inf], [4.8
inf 0.41887903        inf], (4,), float32)
Initial State: [ 0.0273956  -0.00611216  0.03585979  0.0197368 ]
Sample Action Taken: 0
Next State (Transition): [ 0.02727336 -0.20172954  0.03625453  0.32351476]
Reward: 1.0
Terminated: False, Truncated: False
Info: {}

<TimeLimit<OrderEnforcing<PassiveEnvChecker<CartPoleEnv<CartPole-v1>>>>>
```

3.2 Frozen Lake

```
explore_env("FrozenLake-v1")
```

```
============================================================
Environment: FrozenLake-v1
============================================================
Action Space: Discrete(4)
Observation Space: Discrete(16)
```

```
Initial State: 0
Sample Action Taken: 0
Next State (Transition): 0
Reward: 0
Terminated: False, Truncated: False
Info: {'prob': 0.3333333333333333}

<TimeLimit<OrderEnforcing<PassiveEnvChecker<FrozenLakeEnv<FrozenLake-v1>>>>>
```

3.3 Mountain Car

```
explore_env("MountainCar-v0")


============================================================
Environment: MountainCar-v0
============================================================
Action Space: Discrete(3)
Observation Space: Box([-1.2  -0.07], [0.6  0.07], (2,), float32)
Initial State: [-0.4452088  0.       ]
Sample Action Taken: 0
Next State (Transition): [-0.44679132 -0.00158252]
Reward: -1.0
Terminated: False, Truncated: False
Info: {}

<TimeLimit<OrderEnforcing<PassiveEnvChecker<MountainCarEnv<MountainCar-v0>>>>>
```

3.4 Blackjack

```
explore_env("Blackjack-v1")


============================================================
Environment: Blackjack-v1
============================================================
Action Space: Discrete(2)
Observation Space: Tuple(Discrete(32), Discrete(11), Discrete(2))
Initial State: (15, 2, 0)
Sample Action Taken: 1
Next State (Transition): (25, 2, 0)
Reward: -1.0
Terminated: True, Truncated: False
Info: {}

<OrderEnforcing<PassiveEnvChecker<BlackjackEnv<Blackjack-v1>>>>
```

3.5 Taxi

```
explore_env("Taxi-v4")


============================================================
Environment: Taxi-v4
============================================================
```

```
Action Space: Discrete(6)
Observation Space: Discrete(500)
Initial State: 386
Sample Action Taken: 0
Next State (Transition): 486
Reward: -1
Terminated: False, Truncated: False
Info: {'prob': 1.0, 'action_mask': array([0, 1, 0, 1, 0, 0], dtype=int8)}

<TimeLimit<OrderEnforcing<PassiveEnvChecker<TaxiEnv<Taxi-v4>>>>>
```

3.6 Cliff Walking

```
explore_env("CliffWalking-v1")


============================================================
Environment: CliffWalking-v1
============================================================
Action Space: Discrete(4)
Observation Space: Discrete(48)
Initial State: 36
Sample Action Taken: 0
Next State (Transition): 24
Reward: -1
Terminated: False, Truncated: False
Info: {'prob': 1.0}

<OrderEnforcing<PassiveEnvChecker<CliffWalkingEnv<CliffWalking-v1>>>>
```

4. Detailed MDP breakdown — Frozen Lake

Frozen Lake has a discrete, fully known transition model stored in `env.unwrapped.P`. Structure: `P[state][action] = [(probability, next_state, reward, done), ...]`

```
env = gym.make("FrozenLake-v1", is_slippery=True)

n_states = env.observation_space.n
n_actions = env.action_space.n
print("Number of States:", n_states)
print("Number of Actions:", n_actions)

state = 0
action = 1
print(f"\nTransitions from state {state}, action {action}:")
for prob, next_state, reward, done in env.unwrapped.P[state][action]:
    print(f"  Prob={prob}, Next State={next_state}, Reward={reward},
Done={done}")

env.close()

Number of States: 16
Number of Actions: 4
```

```
Transitions from state 0, action 1:
  Prob=0.3333333333333337, Next State=0, Reward=0, Done=False
  Prob=0.3333333333333333, Next State=4, Reward=0, Done=False
  Prob=0.3333333333333337, Next State=1, Reward=0, Done=False
```

5. Running full episodes — CartPole example

Demonstrates the complete state → action → reward → next state loop using a random policy.

```python
env = gym.make("CartPole-v1")

for episode in range(3):
    state, info = env.reset()
    total_reward = 0
    done = False
    step = 0

    print(f"\n--- Episode {episode + 1} ---")
    while not done:
        action = env.action_space.sample()  # random policy
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        total_reward += reward
        step += 1
        state = next_state

    print(f"Episode finished after {step} steps, Total Reward: {total_reward}")

env.close()
```

```
--- Episode 1 ---
Episode finished after 25 steps, Total Reward: 25.0

--- Episode 2 ---
Episode finished after 18 steps, Total Reward: 18.0

--- Episode 3 ---
Episode finished after 10 steps, Total Reward: 10.0
```

MDP Summary Table

Environment	State Space	Action Space	Reward Structure
CartPole	Continuous (4D: cart pos, cart vel, pole angle, pole vel)	Discrete (2: left, right)	+1 per timestep survived
FrozenLake	Discrete (16 grid cells)	Discrete (4: left, down, right, up)	+1 reaching goal, 0 otherwise
MountainCar	Continuous (2D: position, velocity)	Discrete (3: left, no push, right)	-1 per timestep until goal
Blackjack	Discrete (player sum, dealer card, usable ace)	Discrete (2: stick, hit)	+1 win, -1 lose, 0 draw

Environment	State Space	Action Space	Reward Structure
Taxi	Discrete (500 states: taxi pos, passenger, dest)	Discrete (6: move x4, pickup, dropoff)	-1 per step, +20 dropoff, -10 illegal pickup/dropoff
CliffWalking	Discrete (48 grid cells)	Discrete (4: up, right, down, left)	-1 per step, -100 falling off cliff